

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG
TP.HCM

Khoa Công thuật Phần mềm

BÁO CÁO TỔNG HỢP DỰ ÁN MÔN HỌC

Môn: Nhập môn Công nghệ Phần mềm (SE104)

SmartEdu

Hệ thống E-Learning Quản lý Khóa học và Kiểm tra Trực tuyến

Học kỳ: Học kỳ 2 – Năm học 2025–2026
Giảng viên HD: Nguyễn Thị Thanh Trúc
Nhóm: 18
Thành viên: Bùi Trần Trọng Nguyên – MSSV: 24521170
Đinh Quang Nhật – MSSV: 24521248
Hồ Thịnh Phát – MSSV: 24521293

Mục lục

1	Đặt vấn đề & Tóm tắt bài toán	3
1.1	Lý do chọn đề tài	3

1.2	Mục tiêu hệ thống	3
1.3	Phạm vi giải quyết	3
1.4	Đối tượng sử dụng chính	3
2	Phương pháp Khởi gợi Yêu cầu	4
2.1	Quy trình thu thập yêu cầu	4
2.1.1	(a) Phỏng vấn có cấu trúc (Structured Interview)	4
2.1.2	(b) Phân tích nghiệp vụ & Tài liệu miền (Domain Analysis)	4
2.1.3	(c) Phân rã & Ưu tiên hóa yêu cầu (Feature Backlog Refinement)	5
2.2	Kết quả thu được	5
3	Các mô hình Thiết kế Hệ thống	5
3.1	Use Case Diagram – Phạm vi và Chức năng Hệ thống	5
3.2	Class Diagram – Cấu trúc Dữ liệu và Quan hệ Đối tượng	5
3.3	Sequence Diagram – Luồng Tương tác Theo Thời gian	6
3.4	Statechart Diagram – Vòng đời Trạng thái Đối tượng	7
3.5	Mối liên kết giữa các mô hình	8
4	Thảo luận & Bài học Kinh nghiệm	8
4.1	Thiết kế dữ liệu ban đầu bị thiếu thực thể / quan hệ	8
4.2	Xung đột ý kiến khi xây dựng Sequence Diagram	9
4.3	Yêu cầu thay đổi muộn (Late Requirement Change)	9
4.4	Khoảng cách giữa mô hình và hiện thực triển khai	9
5	Kết luận	10

1. Đặt vấn đề & Tóm tắt bài toán

1.1. Lý do chọn đề tài

Trong bối cảnh chuyển đổi số giáo dục ngày càng được đẩy mạnh, các trường đại học vẫn còn phụ thuộc nhiều vào quy trình thủ công, phân mảnh trong việc quản lý khóa học, giao bài tập và tổ chức kiểm tra. Sinh viên đăng ký học phần qua các kênh rời rạc (bảng tin, email, nhóm mạng xã hội); giảng viên phân phát tài liệu học tập và đề thi theo cách truyền thống; kết quả kiểm tra được lưu trữ phân tán, gây khó khăn trong tra cứu và thống kê.

Nhóm lựa chọn đề tài **SmartEdu** nhằm thiết kế một nền tảng e-learning tích hợp, thay thế toàn bộ quy trình trên bằng một hệ thống số hóa thống nhất. Theo tài liệu *Vision & Scope* của nhóm, SmartEdu được định vị là:

“Một trung tâm tương tác giáo dục tập trung, phục vụ sinh viên và giảng viên nội bộ trong giai đoạn đầu, với lộ trình tích hợp cơ sở dữ liệu học thuật bên ngoài, cổng thanh toán chứng chỉ và API hội nghị truyền hình trong các phiên bản tương lai.”

1.2. Mục tiêu hệ thống

Hệ thống SmartEdu hướng đến ba mục tiêu cốt lõi:

1. **Số hóa vòng đời khóa học:** Cho phép giảng viên tạo, quản lý và phân phối nội dung (bài giảng, bài tập) theo từng lớp học và học kỳ.
2. **Tự động hóa quy trình kiểm tra:** Cung cấp môi trường tạo đề, tổ chức thi và chấm điểm trực tuyến với các tùy chọn nâng cao như ngẫu nhiên hóa câu hỏi, giới hạn thời gian và hỗ trợ đa lần thử (*multiple attempts*).
3. **Minh bạch hóa thông tin học tập:** Cung cấp dashboard thống kê theo thời gian thực cho cả sinh viên (lịch sử bài thi, điểm số) lẫn giảng viên (báo cáo lớp, tỷ lệ đạt).

1.3. Phạm vi giải quyết

Phiên bản hiện tại (Release 1) tập trung vào ba phân hệ: **quản lý khóa học**, **kiểm tra trực tuyến** và **quản lý người dùng**, được triển khai dưới dạng ứng dụng desktop (WPF / C#) kết nối với backend Firebase (Firestore + Firebase Authentication).

1.4. Đối tượng sử dụng chính

Hệ thống phục vụ ba nhóm actor chính được trình bày trong Bảng 1.

Bảng 1: Các actor chính và vai trò trong hệ thống SmartEdu

Actor	Vai trò chính trong hệ thống
Sinh viên (Student)	Đăng ký khóa học, xem bài giảng, làm bài kiểm tra, tra cứu lịch sử điểm số
Giảng viên (Teacher)	Tạo và quản lý khóa học, soạn đề thi, xem báo cáo kết quả lớp, nhận xét bài làm
Quản trị viên (Admin)	Quản lý tài khoản người dùng, cấp/thu hồi quyền truy cập, theo dõi hoạt động hệ thống

2. Phương pháp Khởi gợi Yêu cầu

2.1. Quy trình thu thập yêu cầu

Nhóm áp dụng phương pháp *elicitation* đa tầng, kết hợp ba kỹ thuật bổ trợ nhau:

(a) *Phỏng vấn có cấu trúc (Structured Interview)*

Nhóm tổ chức các buổi phỏng vấn trực tiếp với giảng viên bộ môn đóng vai trò **domain expert / khách hàng**. Mỗi buổi phỏng vấn được chuẩn bị trước danh sách câu hỏi tập trung vào ba chủ đề:

- (i) Quy trình hiện tại mà giảng viên đang áp dụng trong quản lý lớp học;
- (ii) Những điểm đau (*pain points*) trong việc tổ chức kiểm tra;
- (iii) Kỳ vọng về hệ thống mới.

Kết quả đầu ra là tập **yêu cầu thô** (*raw requirements*) bao gồm các tính năng ưu tiên cao như: tạo đề thi trắc nghiệm, phân quyền theo vai trò, và xuất báo cáo điểm.

(b) *Phân tích nghiệp vụ & Tài liệu miền (Domain Analysis)*

Nhóm nghiên cứu quy trình đăng ký học phần, tổ chức thi và lưu trữ kết quả hiện hành của nhà trường. Các tài liệu nghiệp vụ thu thập được dùng để xây dựng **business rules**, sau đó được hình thức hóa trong tài liệu `SmartEdu_BusinessRules.docx`. Ví dụ điển hình: quy tắc “*Sinh viên chỉ có thể làm lại bài thi tối đa N lần, trong đó N do giảng viên cấu hình*” — một ràng buộc nghiệp vụ trực tiếp chi phối thiết kế thuộc tính `AllowMultipleAttempts` và `MaxAttempts` trong class `Exam`.

(c) *Phân rã & Ưu tiên hóa yêu cầu (Feature Backlog Refinement)*

Các yêu cầu thô được phân loại thành ba mức ưu tiên (*Must-have / Should-have / Nice-to-have*) theo kỹ thuật **MoSCoW prioritization** và ghi nhận vào `SmartEdu_Features_Backlog.csv`. Tập hợp yêu cầu được hình thức hóa toàn diện trong **Software Requirements Specification (SRS)** (`SmartEdu_SRS.docx`), bao gồm cả yêu cầu chức năng (*functional requirements*) và phi chức năng (*non-functional requirements*: hiệu năng, bảo mật, khả năng mở rộng).

2.2. Kết quả thu được

Sau quá trình elicitation, nhóm thu được bộ tài liệu yêu cầu gồm:

- **Vision & Scope Document:** Phạm vi sản phẩm và định hướng phát triển dài hạn.
- **SRS (Software Requirements Specification):** Đặc tả chi tiết 20+ use case, bao gồm luồng chính, luồng thay thế và điều kiện ngoại lệ.
- **Business Rules Document:** 15+ ràng buộc nghiệp vụ ràng buộc trực tiếp đến thiết kế dữ liệu.
- **Feature Backlog:** Danh sách tính năng có thứ tự ưu tiên cho Release 1 và các phiên bản kế tiếp.

3. Các mô hình Thiết kế Hệ thống

3.1. Use Case Diagram – Phạm vi và Chức năng Hệ thống

Use Case Diagram đóng vai trò bản đồ chức năng của hệ thống, xác định rõ ba actor (Student, Teacher, Admin) và tập hợp các use case tương ứng. Các use case trọng tâm bao gồm:

- **Student:** *Course Registration, Take Quiz, View Quiz History, View Quiz Result Detail.*
- **Teacher:** *Create Course, Create Exam, Manage Questions, View Exam Report, Give Feedback.*
- **Admin:** *Manage Users, Block/Unblock User, View System Dashboard.*

Use Case Diagram là điểm xuất phát để nhóm xác định **ranh giới hệ thống** (*system boundary*) và làm cơ sở triển khai các mô hình tiếp theo.

3.2. Class Diagram – Cấu trúc Dữ liệu và Quan hệ Đối tượng

Class Diagram mô hình hóa cấu trúc tĩnh của hệ thống thông qua 13 lớp nghiệp vụ chính. Bảng 2 tóm tắt vai trò của từng class.

Bảng 2: Các class nghiệp vụ chính trong SmartEdu

Class	Vai trò nghiệp vụ
User	Đại diện cho mọi tài khoản; phân biệt vai trò qua thuộc tính Role
Course	Khóa học/lớp học; liên kết với InstructorId và CourseRegistration
Exam	Bài thi với cấu hình nâng cao (thời gian, số lần thử, điểm đạt)
ExamQuestion	Câu hỏi thi; được tổng hợp bởi Exam qua danh sách QuestionIds
ExamSubmission	Bài làm của sinh viên; lưu câu trả lời (AnswerResponse) và trạng thái (SubmissionStatus)
CourseContent / Lesson	Nội dung bài giảng trong khóa học
Assignment / Submission	Bài tập về nhà và lượt nộp bài tương ứng
Notification	Thông báo hệ thống đến người dùng
Comment	Phản hồi và thảo luận trong khóa học

Các quan hệ kết hợp chính giữa các lớp nghiệp vụ được liệt kê chi tiết bằng danh sách dưới đây để tăng tính trực quan:

- **Course 1 - * Exam:** Một khóa học chứa nhiều bài thi.
- **Exam 1 - * ExamQuestion:** Một bài thi chứa nhiều câu hỏi thi.
- **Exam 1 - * ExamSubmission:** Một bài thi có nhiều lượt làm bài từ sinh viên.
- **ExamSubmission 1 - * AnswerResponse:** Một lượt làm bài lưu trữ nhiều câu trả lời tương ứng.

Đây là cơ sở dữ liệu quan trọng để thiết kế cấu trúc Firestore collections tương ứng trong hệ thống.

3.3. Sequence Diagram – Luồng Tương tác Theo Thời gian

Sequence Diagram hiện thực hóa từng use case cụ thể bằng cách mô tả trình tự trao đổi thông điệp giữa các đối tượng. Hai luồng tương tác cốt lõi được mô tả chi tiết từng bước như sau:

1. Luồng “Sinh viên làm bài thi” (Take Quiz):

- StudentQuizView gửi yêu cầu lấy thông tin đề thi.
- Hệ thống gọi hàm `FirestoreService.GetExam()` để truy vấn dữ liệu từ Firestore.

- Firestore trả về thông tin đối tượng `Exam` kèm danh sách các mã câu hỏi `QuestionIds`.
- Giao diện `TakeQuizView` tải câu hỏi chi tiết từ cơ sở dữ liệu và hiển thị để sinh viên làm bài.
- Sinh viên hoàn tất bài thi, thực thể `ExamSubmission` được khởi tạo để lưu thông tin bài làm.
- Hệ thống gọi hàm `FirestoreService.SubmitExam()` để lưu trữ kết quả và chấm điểm tự động.
- Giao diện `QuizResultDetailView` tải và hiển thị kết quả bài làm chi tiết cho sinh viên.

2. Luồng “Giảng viên tạo đề thi” (Create Exam):

- Giao diện `CreateExamView` tiếp nhận dữ liệu cấu hình bài thi và thực hiện kiểm tra tính hợp lệ (validation).
- Đối tượng tạm `ExamDraft` được khởi tạo trong bộ nhớ.
- Giao diện `CreateExamQuestionsView` cho phép giảng viên biên soạn nội dung các câu hỏi.
- Hệ thống gọi hàm `FirestoreService.SaveExam()` để đồng bộ và lưu trữ đề thi hoàn chỉnh lên Firestore.

Sequence Diagram giúp nhóm phát hiện các **điểm đồng bộ hóa dữ liệu** tiềm ẩn (ví dụ: trạng thái `IsPublished` phải được cập nhật đồng thời với `QuestionIds`) mà Class Diagram không thể hiện được.

3.4. Statechart Diagram – Vòng đời Trạng thái Đối tượng

Statechart Diagram được xây dựng cho hai thực thể có vòng đời trạng thái phức tạp nhất:

Vòng đời *ExamSubmission*:

```
[Khoi tao] --> InProgress --> Submitted --> Graded
                                     \--> Expired (het thoi
                                           gian)
```

Diagram này ánh xạ trực tiếp đến enum `SubmissionStatus {InProgress, Submitted, Graded, Expired}` trong code và xác định các **guard conditions**: chuyển từ `Submitted` sang `Graded` chỉ xảy ra khi giảng viên nhấn “Chấm điểm” hoặc hệ thống tự động tính điểm với câu hỏi trắc nghiệm.

Vòng đời *Exam*:

```
[Draft] --> (Published, IsActive=true) --> Ongoing --> Closed
                                     \--> Cancelled (giang
                                           vien thu hoi)
```

Statechart này lý giải tại sao một bài thi đã **Published** không thể chỉnh sửa câu hỏi mà không thu hồi về trạng thái **Draft** trước.

3.5. Mối liên kết giữa các mô hình

Các mô hình không tồn tại độc lập mà có quan hệ phụ thuộc theo luồng từ trừu tượng đến cụ thể:

```
Use Case Diagram    -- "He thong lam gi?"
    |
    v
Class Diagram        -- "Du lieu duoc cau truc the nao?"
    |
    v
Sequence Diagram    -- "Cac doi tuong tuong tac ra sao de thuc
    hien use case?"
    |
    v
Statechart Diagram  -- "Doi tuong thay doi trang thai nhu the
    nao qua vong doi?"
```

Ví dụ minh họa chuỗi truy xuất: Use case *Take Quiz* → (Đặc tả SRS) → Sequence Diagram → (Sử dụng) các class *Exam*, *ExamSubmission*, *AnswerResponse* (từ Class Diagram) → (Quản lý trạng thái) → Statechart của *ExamSubmission*.

4. Thảo luận & Bài học Kinh nghiệm

4.1. Thiết kế dữ liệu ban đầu bị thiếu thực thể / quan hệ

Khó khăn. Trong phiên bản Class Diagram đầu tiên, nhóm chỉ mô hình hóa *Exam* và *User* mà bỏ qua thực thể trung gian *ExamSubmission*. Hệ quả là không có nơi lưu trữ câu trả lời từng câu hỏi của sinh viên (*AnswerResponse*), khiến chức năng “*View Quiz Result Detail*” không thể thiết kế được ở bước Sequence Diagram.

Giải quyết. Nhóm quay lại phân tích use case *View Quiz Result Detail* từ góc độ dữ liệu cần hiển thị, từ đó truy ngược ra các thực thể còn thiếu. Đây là minh chứng cho nguyên

tắc **iterative modeling**: các mô hình phải được làm mịn qua nhiều vòng (*refinement cycles*), không thể hoàn thiện ngay từ đầu.

Bài học. Khi thiết kế Class Diagram, cần đặt câu hỏi “*Dữ liệu này được hiển thị ở đâu và ai cần đọc nó?*” cho mỗi thực thể, thay vì chỉ mô hình hóa theo trực giác.

4.2. Xung đột ý kiến khi xây dựng Sequence Diagram

Khó khăn. Khi đặc tả luồng nộp bài, nhóm phát sinh tranh luận: “*Điểm số nên được tính phía client (ứng dụng WPF) hay phía server (Cloud Function)?*” Hai quan điểm song song dẫn đến hai phiên bản Sequence Diagram tồn tại đồng thời trong một thời gian.

Giải quyết. Nhóm tổ chức review session, so sánh hai phương án theo ba tiêu chí: (i) tính nhất quán dữ liệu (*data integrity*), (ii) khả năng gian lận (*cheating prevention*), (iii) độ phức tạp triển khai. Phương án tính điểm phía server được chọn vì đảm bảo tốt hơn hai tiêu chí đầu.

Bài học. Xung đột trong quá trình modeling là bình thường và **có giá trị** — nó buộc nhóm phải tư duy rõ ràng hơn về trade-off thiết kế. Cần có tiêu chí đánh giá tường minh để phân giải xung đột, tránh quyết định dựa trên cảm tính.

4.3. Yêu cầu thay đổi muộn (Late Requirement Change)

Khó khăn. Sau khi Class Diagram và Sequence Diagram đã hoàn thiện, khách hàng yêu cầu bổ sung tính năng “*Giảng viên có thể để lại nhận xét trên từng bài làm*”. Điều này yêu cầu cập nhật `ExamSubmission` (thêm thuộc tính `FeedbackFromTeacher`, `GradedAt`) và điều chỉnh Sequence Diagram luồng chấm điểm.

Giải quyết. Nhóm xử lý thay đổi theo nguyên tắc **impact analysis**: liệt kê tất cả mô hình và module bị ảnh hưởng, sau đó cập nhật tuần tự theo thứ tự Class Diagram → SRS → Sequence Diagram → code. Nhờ đó tránh được tình trạng *model-code divergence*.

Bài học. Yêu cầu thay đổi muộn là không thể tránh khỏi trong thực tế. Hệ thống tài liệu nhất quán (SRS, models) giúp đánh giá **phạm vi ảnh hưởng** (*impact scope*) nhanh chóng và chính xác, từ đó giảm chi phí thay đổi.

4.4. Khoảng cách giữa mô hình và hiện thực triển khai

Khó khăn. Statechart Diagram mô tả trạng thái `Cancelled` của `Exam`, nhưng Firebase Firestore không hỗ trợ atomic state transitions — việc cập nhật đồng thời `IsActive = false` và xóa `QuestionIds` phải được xử lý thủ công trong transaction.

Giải quyết. Nhóm bổ sung **implementation notes** vào Sequence Diagram để ghi nhận rằng buộc kỹ thuật này, giúp tách biệt rõ logic nghiệp vụ (*business logic*) khỏi ràng buộc công nghệ (*technology constraint*).

Bài học. Mô hình UML thể hiện **what** (hệ thống làm gì), không phải **how** (làm thế nào ở mức kỹ thuật). Khi triển khai, cần thêm lớp *implementation notes* để lấp đầy khoảng cách này.

5. Kết luận

Dự án SmartEdu đã hoàn thành mục tiêu Release 1 với mức độ phủ chức năng đáp ứng tốt yêu cầu ban đầu. Bảng 3 tổng hợp mức độ hoàn thành theo từng mục tiêu.

Bảng 3: Mức độ hoàn thành các mục tiêu Release 1

Mục tiêu ban đầu	Trạng thái	Ghi chú
Quản lý khóa học & lịch giảng	Hoàn thành	Tạo/sửa/xóa khóa học; Student đăng ký
Hệ thống kiểm tra trực tuyến	Hoàn thành	Đầy đủ luồng tạo đề → làm bài → chấm điểm → xem kết quả
Phân quyền 3 vai trò	Hoàn thành	Admin / Teacher / Student với UI riêng biệt
Báo cáo & thống kê	Hoàn thành	ExamReport (Teacher), Dashboard (Admin & Student)
Tích hợp video conferencing	Chuyển sang Release 2	Trong roadmap <i>Vision & Scope</i>

Về phương diện phân tích thiết kế, bốn mô hình UML (Use Case, Class, Sequence, Statechart) khi được xây dựng có hệ thống và theo đúng thứ tự phụ thuộc đã hỗ trợ hiệu quả cho quá trình triển khai: giảm thiểu sai sót về logic nghiệp vụ, phát hiện sớm các yêu cầu mâu thuẫn và cung cấp ngôn ngữ chung (*common language*) để nhóm thống nhất ý kiến.

Kinh nghiệm quan trọng nhất rút ra là: **phân tích thiết kế không phải là giai đoạn tuyến tính một chiều, mà là một quá trình lặp liên tục** (*iterative process*) — trong đó mỗi mô hình mới xây dựng đều có thể phản hồi ngược lại và làm mịn các mô hình trước đó. Đây chính là bản chất của *Requirement Refinement* trong thực tế kỹ nghệ phần mềm.